

From Waterfall to Agile: the Manifesto and the mindset

DIG 123 – Agile Methodologies for Innovation (3 credits) · Class 2 of 15 · Sprint 0

Monday, September 28, 2026 · Lecture hour (60 minutes)

Lecture hour · four lessons, four activities · Cyber.SoHo Educational Hub



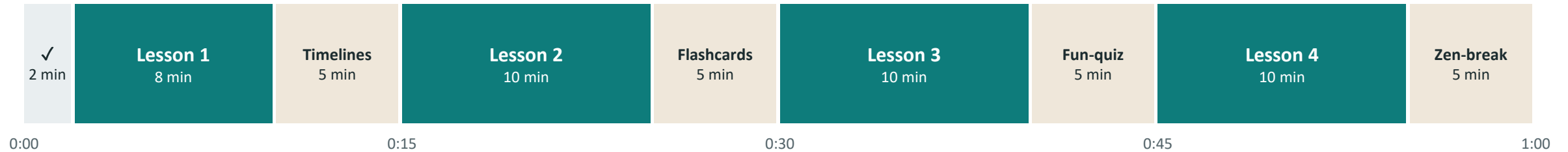
From the Industry to the Classroom — Building the IT's Future, Today and Together!

Educational material — © 2026 Cyber.SoHo. All rights reserved. Use is subject to the Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy ([link on the last slide](#)).

Required text: Denning, S. (2018). *The Age of Agile: How Smart Companies Are Transforming the Way Work Gets Done*. AMACOM (the book-publishing division of the American Management Association).

Today's 60 minutes

Class 2 · Sep 28, 2026



- 0:00 Check-in (2 min) — attendance in Blackboard, phones out
- 0:02 Lesson 1 · The problem with big plans (8 min) → visualization: two timelines (5)
- 0:15 Lesson 2 · The four values (10) → flashcards ×4, call and response (5)
- 0:30 Lesson 3 · Twelve principles, four themes (10) → fun-quiz ×3 (5)
- 0:45 Lesson 4 · Denning's three laws (10) → zen-break + one-minute paper (5)
- 1:00 Team Studio (120 min) · then the Home Sprint (120 min)



0:00 · the 60-minute plan · 1:00 — the rust block is where we are



By the end of this hour you can

Class 2 · Sep 28, 2026

1

LESSON 1

Explain why a plan made at the start of a project carries the largest error — the **cone of uncertainty** — and say what a **Waterfall** life cycle is and where it comes from

2

LESSON 2

State the four values of the **Agile Manifesto** and explain what "over" means: a priority, not an exclusion

3

LESSON 3

Group the twelve principles into four themes, recognize a broken principle, and define **iteration, increment** and **feedback loop**

4

LESSON 4

State Denning's three laws — the Small Team, the Customer, the Network — and give one example of Agile outside software

- Syllabus alignment: Learning Outcome 1 — understand the concept and utility of Agile methodologies in business applications
- Reading: Denning (2018), *The Age of Agile* — Chapter 1 for today, Chapter 2 for next week (your copy is in VitalSource)



0:00 · the 60-minute plan · 1:00 — the rust block is where we are



Lesson 1 · The problem with big plans

0:02 – 0:10 · then the activity: visualization — two timelines (5 min)

Where the sequential life cycle comes from, why the estimate made on day one is the worst one you will ever make, and what seventeen practitioners did about it in February 2001.

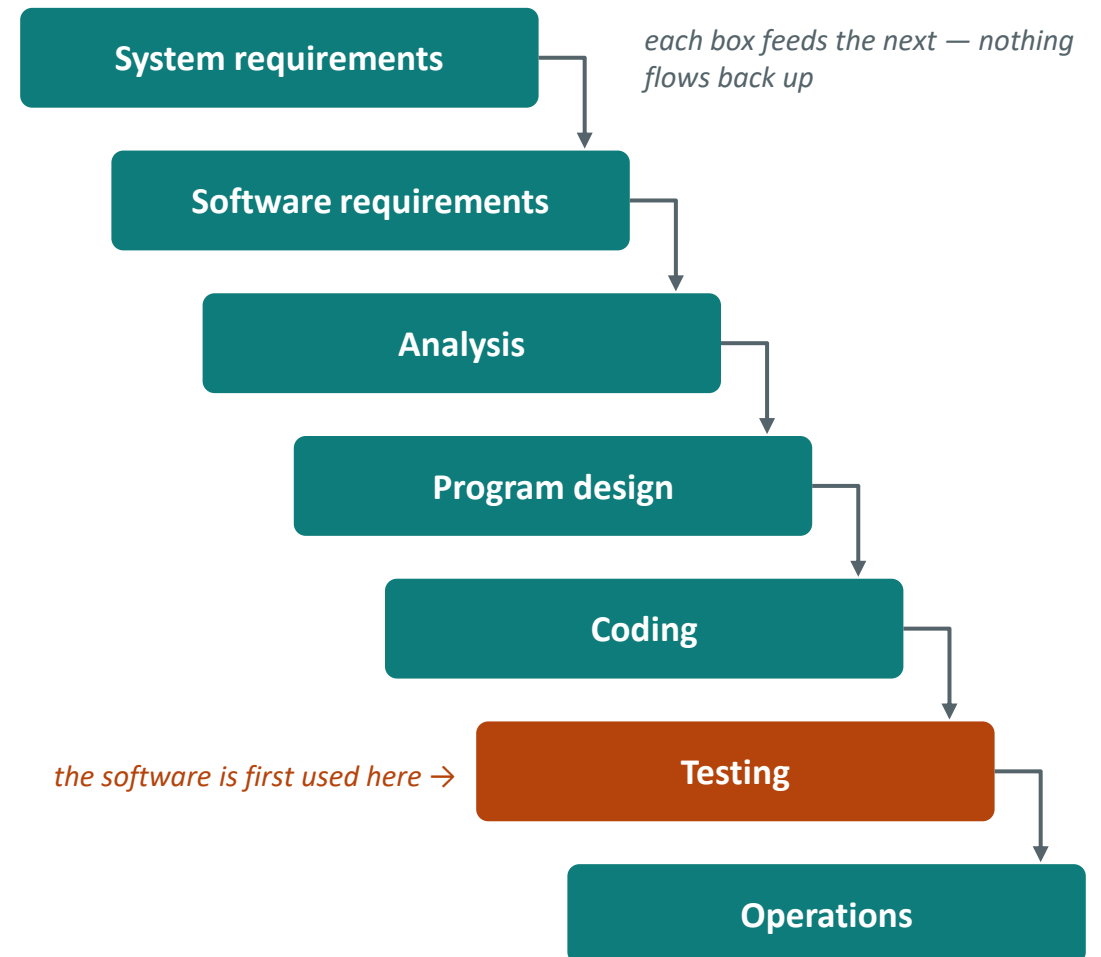


0:00 · the 60-minute plan · 1:00 — the rust blocks are this lesson and its activity



Where the sequential life cycle comes from (1/2)

- Last Monday: a line from "plan everything first" to "learn as you go" — today, its left end, up close
- **1970** — Winston W. Royce, *Managing the Development of Large Software Systems*
- Seven boxes in a staircase: each box feeds the next, and nothing flows back up
- **Waterfall:** a sequential life cycle — each phase is completed and approved before the next begins
- The word "waterfall" is not in the paper: later authors attached it to the drawing



Where the sequential life cycle comes from (2/2)

What Royce wrote

- The simple sequence "is risky and invites failure": testing — the first moment the software is actually used — comes last, after every earlier decision has been paid for.
- His advice: run the sequence at least twice, with a preliminary version first.



What the industry did

- It adopted the drawing and ignored the advice.
- 1985 — **DOD-STD-2167** (Department of Defense Standard 2167) was read as "work in phases, approve a document at the end of each".
- The United States defence budget paid for a large share of the world's software → the sequence became the default for a generation.



The cone of uncertainty — a definition

- **Cone of uncertainty:** a model of how the accuracy of an estimate changes over the life of a project
- Measured by Barry Boehm, *Software Engineering Economics* (1981), from data on hundreds of projects
- Named and popularized by Steve McConnell; his company Construx keeps the reference description
- At the idea (feasibility) stage the honest estimate is a **range**, not a number:
 - from $\frac{1}{4}$ of the eventual actual value to $4 \times$ the actual value

$\times / \div 4$

the honest range on day one

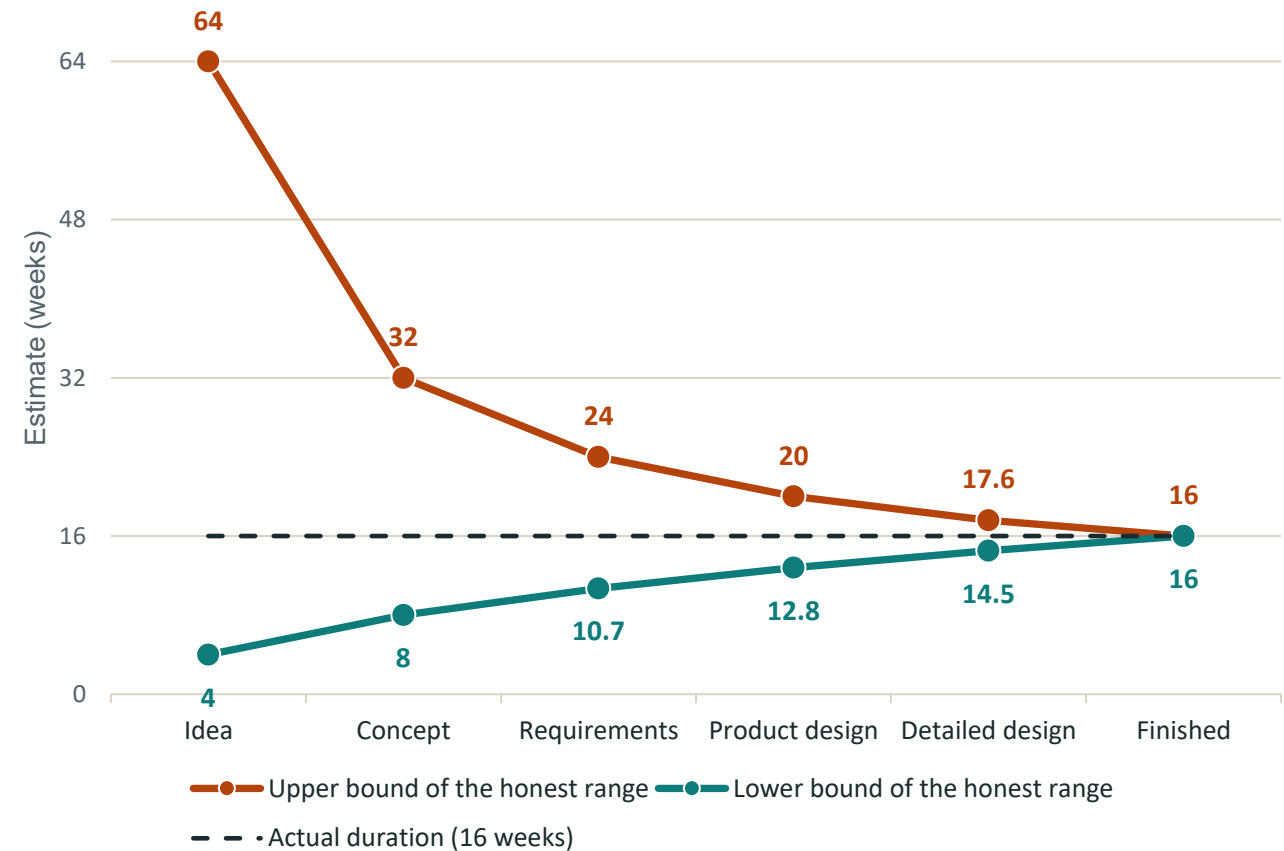
A 16-week project, estimated at the idea stage: anywhere from 4 to 64 weeks



The cone narrows — milestone by milestone

- Each decision that removes a source of variation narrows the range:
 - concept of operation agreed $\rightarrow \times/\div 2$
 - requirements accepted $\rightarrow \times/\div 1.5$
 - product design done $\rightarrow \times/\div 1.25$
 - detailed design done $\rightarrow \times/\div 1.1$
 - work finished $\rightarrow$ the actual value
- On a chart: wide on the left, closing to a point on the right — hence the name

A 16-week project: the honest range (weeks) at each milestone



Why big plans fail — two consequences

1 · The cone does not narrow by itself

- Time alone changes nothing. The cone narrows only when a decision or information removes variability: a scope decision, a prototype that proves a technology, a user who confirms a feature.
- Six months of planning without learning = six months at $\times/\div 4$.

2 · The commitment sits at the widest point

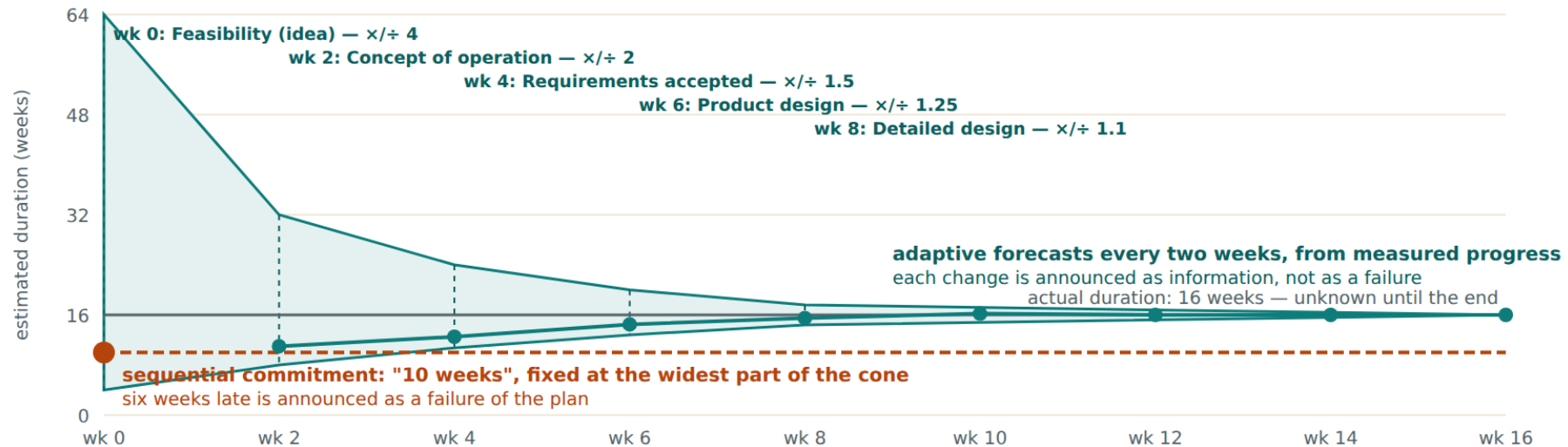
- A sequential life cycle asks for the binding commitment — budget, deadline, signed feature list — at the left of the cone, at maximum error...
- ...and then forbids changing it: every later discovery can only be a failure of the plan, never information.



Where each plan commits on the cone

The cone of uncertainty — one commitment made early, one made in steps

Boehm (1981), as described by McConnell / Construx. A project that will actually take 16 weeks; the band is the honest range at each milestone.



The cone narrows only when a decision or a measurement removes variability; time alone does not narrow it.

A sequential plan fixes budget, date and scope at $\times/\div 4$. An adaptive plan commits late, in small steps, at $\times/\div 1.25$ or better.

Factors: Boehm, B. (1981), Software Engineering Economics; McConnell, S., Construx, "The Cone of Uncertainty".

Not a moral failing of the people involved — a property of where the commitment sits on the chart. The sequential plan commits (rust line) at the widest point and must defend that promise; the adaptive plan re-forecasts (teal dots) every two weeks from measured progress, so its error shrinks as the cone does.

The evidence: the CHAOS reports

- The Standish Group (a research firm in Massachusetts) has published the **CHAOS** report — the study's name, not an abbreviation — since 1994
- Built from surveys of thousands of software projects
- Projects run with small, adaptive methods succeed more often
- The larger the project, the worse the numbers for the sequential approach
- Remember the mechanism, not the percentages: **commitment at maximum uncertainty, no correction allowed**

≈ 1 in 3

projects finish on time, on budget and
with the planned scope

*Edition after edition of the CHAOS
reports*



February 11–13, 2001 — Snowbird, Utah

Lesson 1 · 0:02–0:10

The "lightweight" methods of the 1990s

XP

(Extreme Programming)
Kent Beck

Scrum

Jeff Sutherland & Ken Schwaber,
1995

DSDM

(Dynamic Systems Development
Method)
United Kingdom

FDD

(Feature-Driven Development)

Crystal

Alistair Cockburn

Adaptive Software Development

Jim Highsmith

- Seventeen practitioners of these "lightweight" methods met at the Snowbird ski resort (Wasatch mountains, Utah); Robert C. Martin sent the invitations
- They expected to disagree. In two days they agreed on a name — **Agile**, in place of "lightweight" — and on four sentences: the **Manifesto for Agile Software Development**
- The twelve principles followed by e-mail; later in 2001 the same group founded the non-profit **Agile Alliance**
- Unchanged since, in 60+ languages: agilemanifesto.org — Box 1-1 in Denning



0:00 · the 60-minute plan · 1:00 — the rust block is where we are



What the Manifesto is not

- **Not a method:** it prescribes no meeting, no role, no document
- A statement of what the seventeen valued **more** — written so that it still keeps what they valued less
- Its construction: "this **over** that" — not "this instead of that"
- That one word, "over", is the subject of Lesson 2

Not a method — it prescribes no...

~~meetings~~

~~roles~~

~~documents~~

It states a priority:

this **OVER** that

this ~~INSTEAD OF~~ that



Activity · Visualization: two timelines (5 min)

- One sheet, two lines, fourteen weeks each — for your team's product idea from Class 1
- Top line — a **Waterfall** plan: requirements, design, build, test, release, with the week each one ends
- Bottom line — an **Agile** plan: two-week iterations and the usable piece finished at the end of each
- Put a star wherever a real user sees and tries something; count the stars on each line
- 2 min to draw · 1 min to count · 2 min: three volunteers show their sheet and say the two numbers
- Keep the sheet: it goes into **D2** (Deliverable 2)

5

minutes

0:10 – 0:15

taking part earns activity points

Videos for this lesson (each under 15 minutes):

- **Video:** [What is the Cone of Uncertainty? Project Management in Under 5](https://youtube.com/watch?v=QZHQv88kYgw) youtube.com/watch?v=QZHQv88kYgw
- **Video:** [How The Agile Manifesto Came To Be](https://youtube.com/watch?v=AGdi3zZutXk) youtube.com/watch?v=AGdi3zZutXk



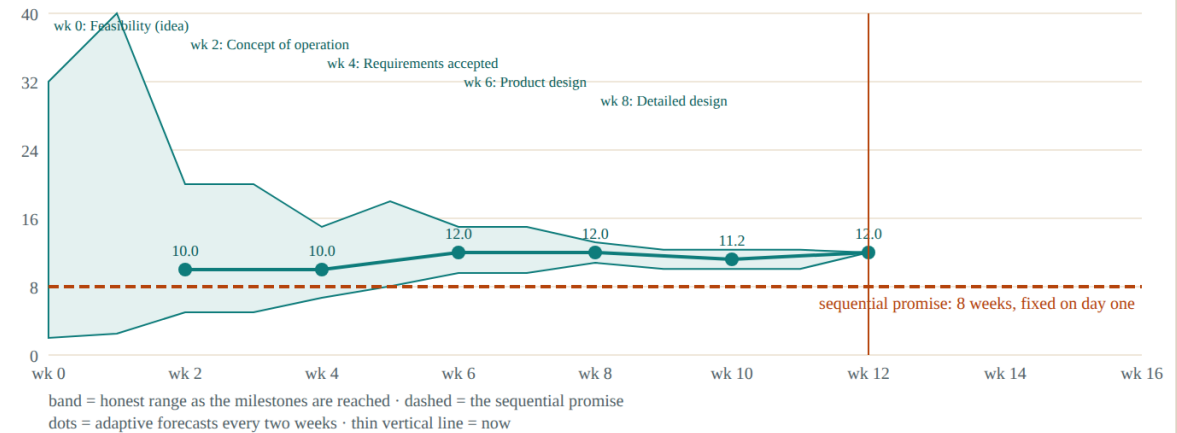
0:00 · the 60-minute plan · 1:00 — the rust block is where we are



Real-life example 1 · One estimate, two commitments

- Team Menu builds a cafeteria-menu page with allergen information and must say when it will be ready
- Sequential path: commits on day one — 100 units ÷ 12.5 units per week = **8 weeks**, scope fixed
- Two discoveries: week 5, a supplier feed needs a connector (+20 units); week 9, a feature nobody asked for is dropped (−8)
- Adaptive path: re-forecasts every two weeks from the measured rate (10 units per week): 10 → 12 → 11.2 weeks
- Finished in week 12: the promise missed by 4 weeks — a failure of the plan; the adaptive path had shown week 12 since week 6

Estimated duration (weeks) as the project advances — cone, promise, forecasts



Final state of the simulation · run it live in the lecture page (Start / Pause · Next step · Restart · Copy)

Where it returns in the labs: D2 (Deliverable 2) plan comparison · D5 (Deliverable 5) release forecast · Mid-Term Part B

Lesson 1 in six cards

Lesson 1 · 0:02–0:10

Who drew the staircase that became "Waterfall", and when?

Winston W. Royce, 1970, in *Managing the Development of Large Software Systems* — and he warned that a single pass through it "invites failure".

What is the cone of uncertainty?

A model of estimate accuracy: at the idea stage the honest estimate is a range of $\times/\div 4$ around the eventual actual; it narrows (2, 1.5, 1.25, 1.1) only as decisions and information remove variability.

Where does a sequential plan place its commitment?

At the widest part of the cone — day one, maximum error — and then forbids changing it.

Does the cone narrow with time alone?

No. It narrows only when the team makes decisions or obtains information that remove sources of variation.

What happened at Snowbird, February 11–13, 2001?

Seventeen practitioners of "lightweight" methods agreed on the word Agile and wrote the four-sentence Manifesto for Agile Software Development; the twelve principles followed by e-mail.

What is the Manifesto not?

Not a method: it prescribes no meeting, role or document. It states what its authors value more, while keeping what they value less.



0:00 · the 60-minute plan · 1:00 — the rust block is where we are



Lesson 2 · The four values — "over", not "instead of"

0:15 – 0:25 · then the activity: flashcards ×4, call and response (5 min)

The four sentences of the Manifesto read one by one: what each left side means, why each right side still has value, and how to recognize a value in a team's behaviour.

✓	L1	act	L2	act	L3	act	L4	act
---	----	-----	----	-----	----	-----	----	-----

0:00 · the 60-minute plan · 1:00 — the rust blocks are this lesson and its activity



The Manifesto, word for word

Lesson 2 · 0:15–0:25

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

© 2001, the seventeen authors listed at agilemanifesto.org. This declaration may be freely copied in any form, but only in its entirety through this notice. Source: agilemanifesto.org



The grammar: "over" means priority

PRIORITY — what it asks

- When the two sides conflict — time, money or attention for only one — the left side wins.
- The right side keeps its value and is still done.
- The closing sentence says so: *while there is value in the items on the right, we value the items on the left more.*

≠

EXCLUSION — the error

- No documentation, no contract, no plan = the Manifesto misread, not applied.
- The most common misreading in the industry — and the reason the closing sentence exists.
- Keep the two words apart all semester: **priority** is what the Manifesto asks for; **exclusion** is the error.



Value 1 · Individuals and interactions over processes and tools

LEFT SIDE · VALUED MORE

Individuals and interactions

The people doing the work and their direct exchanges — a conversation, a question answered at the next desk, a decision taken together

Why valued more: Understanding is produced by interaction: a process cannot notice that two people mean different things by the same word; a five-minute conversation can

over

RIGHT SIDE · STILL HAS VALUE

processes and tools

Defined procedures and the software used to follow them — a ticketing system, an approval workflow, a template

Why it still has value: A process makes work repeatable; a tool keeps a record that survives the conversation

In practice → When the tool and the conversation disagree, fix the understanding first — then update the tool



Value 2 · Working software over comprehensive documentation

Lesson 2 · 0:15–0:25

LEFT SIDE · VALUED MORE

Working software

A product that a user can actually operate, however small

Why valued more: Working software is evidence; a document is a prediction. A working feature in front of a user settles: does it work, is it wanted, is it understood the same way?

over

RIGHT SIDE · STILL HAS VALUE

comprehensive documentation

Complete written descriptions of what the product will do, how it is built and how it is used

Why it still has value: Needed documentation — acceptance criteria, interfaces, instructions — is written, and written well. The objection is to "comprehensive"

In practice → From Class 4 the rule becomes your Definition of Done: "done" means usable, including the documentation that makes it usable



0:00 · the 60-minute plan · 1:00 — the rust block is where we are



Value 3 · Customer collaboration over contract negotiation

LEFT SIDE · VALUED MORE

Customer collaboration

Working with the person who will use or pay for the product — continuously, as it takes shape

Why valued more: A contract is negotiated at the widest part of the cone: it fixes both parties' shared ignorance. Collaboration lets them use what they learn

over

RIGHT SIDE · STILL HAS VALUE

contract negotiation

Agreeing in advance, in a binding document, exactly what will be delivered, when and for how much

Why it still has value: A contract protects both parties from bad faith; Agile contracts fix the budget and the cadence and leave the detailed scope open

In practice → The customer attends every iteration review and can re-order the remaining work — in Scrum, the Product Owner (Class 3)



0:00 · the 60-minute plan · 1:00 — the rust block is where we are



Value 4 · Responding to change over following a plan

LEFT SIDE · VALUED MORE

Responding to change

Treating a change in requirements, technology or market as information, and adjusting the work to it

Why valued more: A plan is a hypothesis made under uncertainty; followed against reality, it yields a product that fits the plan, not the need

over

RIGHT SIDE · STILL HAS VALUE

following a plan

Executing the approved plan and measuring success as conformance to that plan

Why it still has value: Without a plan there is nothing to adapt, nobody can coordinate with the team, and a change cannot even be seen as a change

In practice → Plan constantly, in short horizons (a Sprint, a release), and re-plan at fixed intervals from what was actually finished



0:00 · the 60-minute plan · 1:00 — the rust block is where we are

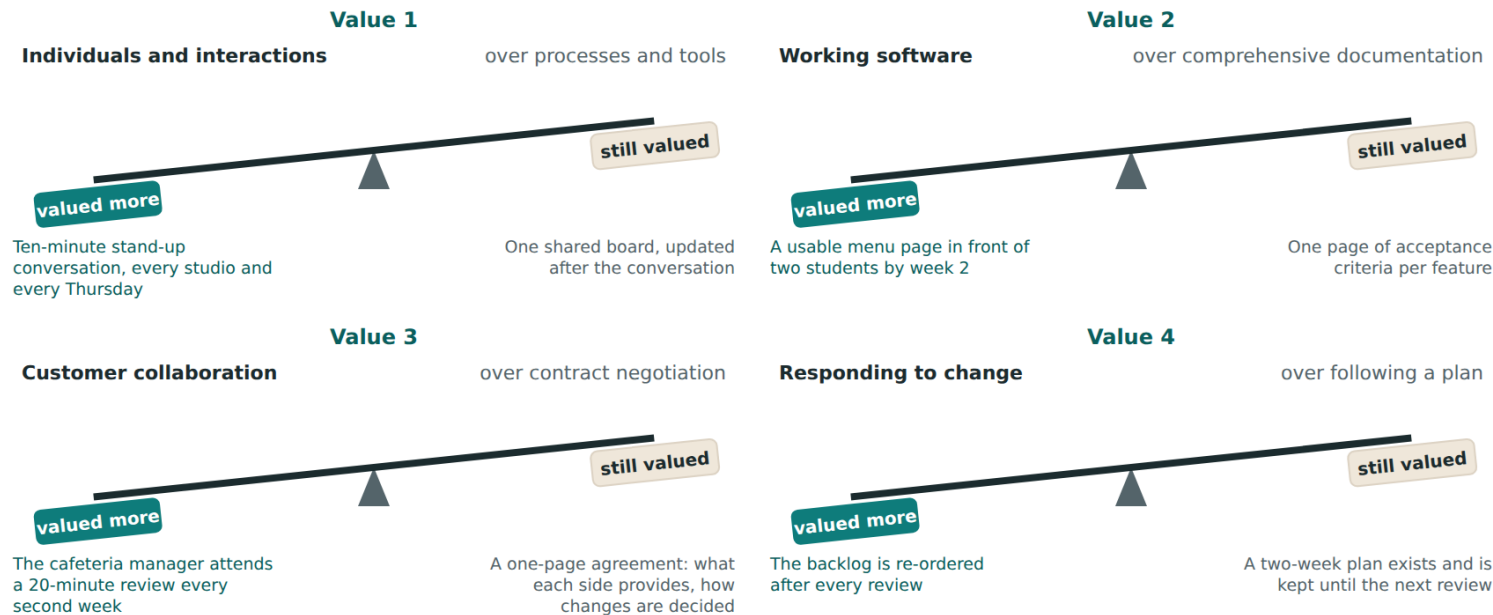


Four values, four scales — reading them together

Lesson 2 · 0:15–0:25

"Over", not "instead of" — four scales that tilt left and keep their right pan

Manifesto for Agile Software Development (2001), read with its closing sentence:
"while there is value in the items on the right, we value the items on the left more."



A scale that tilts fully left with an empty right pan is not the Manifesto applied; it is the Manifesto misread.
Team Menu's map keeps something light on every right pan — the shape of the Manifesto map your team hands in with D2.

WHAT THE DIAGRAM SHOWS

Every scale tilts left and keeps something on its right pan. The four right sides — a tool, a document, a contract, a plan — can be inspected and controlled from outside, in advance; the four left sides exist only between people while the work is done. Denning calls the shift a change of mindset, not of method.



Your Manifesto map for D2

- For each value, one practice your team commits to: an **observable behaviour with a frequency**
- Not a slogan — "we demonstrate the increment to two real users at the end of every iteration", not "we listen to customers"
- Part of **D2** (Deliverable 2); in Class 3 it becomes your values pledge in **D3** (Deliverable 3)

Value	Team Menu — the practice (left side)	What stays on the right
1 · Individuals and interactions	A ten-minute stand-up conversation every studio and every Thursday	One shared board, updated after the stand-up, visible to the manager
2 · Working software	A usable menu page in front of two students by week 2	One page of acceptance criteria per feature, written before the feature
3 · Customer collaboration	The cafeteria manager attends a 20-minute review every second week	A one-page agreement: what the manager provides, what the team promises, how changes are decided
4 · Responding to change	The backlog is re-ordered after every review	A two-week plan exists and is kept until the next review



Activity · Flashcards ×4, call and response (5 min)

- Round 1, in order — the professor says the left side of a value, the whole class answers the right side
- Round 2, reversed — the right side first, the class answers the left
- Round 3, shuffled — the professor says a right side; one student says in one sentence why it **still has value**
- Four values, four students, four sentences

5

minutes

0:25 – 0:30

taking part earns activity points

Videos for this lesson (each under 15 minutes):

- **Video:** [Agile Manifesto Explained | The 4 Core Values of Agile \(Scrum, XP & More\)](https://youtube.com/watch?v=eglpPk6oAis) youtube.com/watch?v=eglpPk6oAis
- **Video:** [The Agile Manifesto Explained - 4 Values](https://youtube.com/watch?v=MnJDdzxgeR0) youtube.com/watch?v=MnJDdzxgeR0

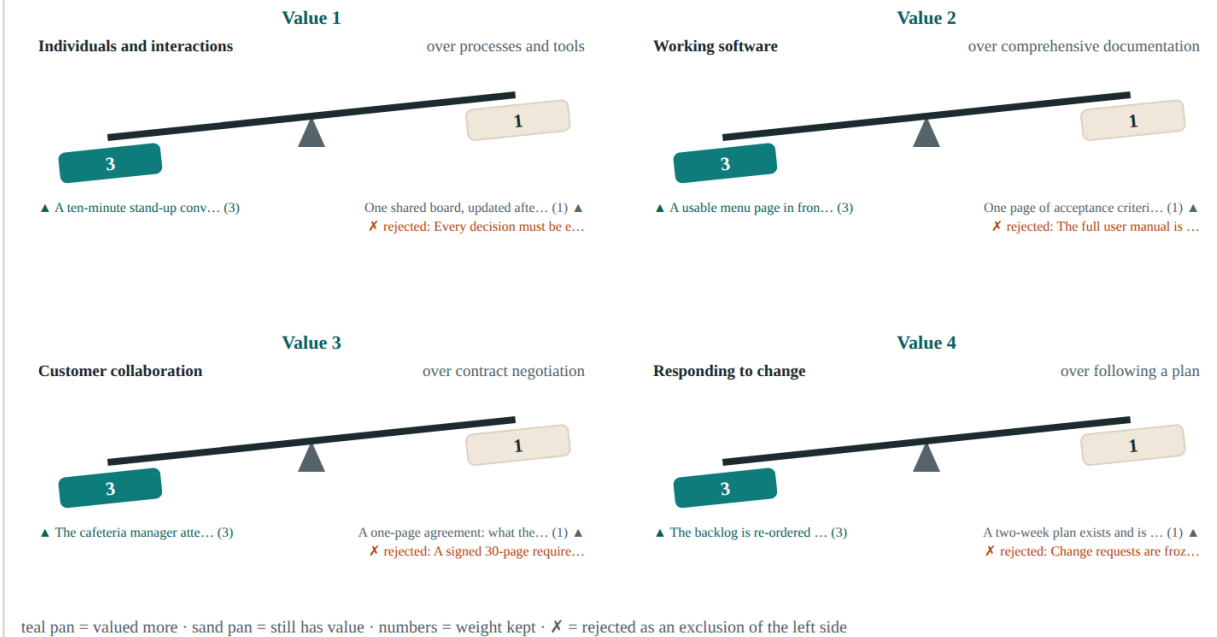


Real-life example 2 · Team Menu's Manifesto map

Lesson 2 · 0:15–0:25

- Team Menu lists twelve candidate practices for its D2 map
- Each one: which value? which side? how heavy (1–3)? kept or rejected?
- Kept practices tilt four scales to the left
- Four are rejected — each applies a right side as an exclusion: a signed 30-page requirements document before any work · every decision valid only once it is in the ticketing tool · change requests frozen after week 1 · the full user manual before any coding
- The finished map: every scale tilts left; every right pan still holds something light

Four values, four scales — the beam tilts toward what is valued more



Final state of the simulation · run it live in the lecture page (Start / Pause · Next step · Restart · Copy)

Where it returns in the labs: D2 (Deliverable 2) Manifesto map · D3 (Deliverable 3) values pledge · D4 (Deliverable 4) Definition of Done · Mid-Term Part A



Lesson 2 in six cards

Lesson 2 · 0:15–0:25

What does the word "over" mean in the Manifesto?

A priority: when the two sides conflict, the left wins — while the right side keeps its value and is still done. Priority, not exclusion.

Value 1: what is valued over what, and why?

Individuals and interactions over processes and tools — because understanding between people is produced by interaction; a process cannot notice a misunderstanding.

Value 2: what is valued over what, and why?

Working software over comprehensive documentation — working software is evidence, a document is a prediction; documentation that is needed is still written.

Value 3: what is valued over what, and why?

Customer collaboration over contract negotiation — a contract fixes both parties' knowledge at the widest part of the cone; collaboration lets them use what they learn.

Value 4: what is valued over what, and why?

Responding to change over following a plan — a plan is a hypothesis; Agile teams plan in short horizons and re-plan at fixed intervals.

What is a Manifesto map, and where does it go?

For each value, one observable practice with a frequency that your team commits to — part of D2 (Deliverable 2), due Sunday, October 4.



0:00 · the 60-minute plan · 1:00 — the rust block is where we are



Lesson 3 · Twelve principles, four themes

0:30 – 0:40 · then the activity: fun-quiz ×3 — which principle is broken? (5 min)

The principles as behaviours you can observe in a team, grouped into four themes; then the three words that make them possible — iteration, increment, feedback loop — and the Sprint that contains them.

✓	L1	act	L2	act	L3	act	L4	act
---	----	-----	----	-----	----	-----	----	-----

0:00 · the 60-minute plan · 1:00 — the rust blocks are this lesson and its activity



From values to behaviours: twelve principles, four themes

- The values say what the seventeen valued; the principles say what a team that holds those values **does**
- Your job: recognize each principle in behaviour — what the fun-quiz, Checkpoint Quiz 1 and the Mid-Term test
- The four themes are this course's grouping; the sentences are the Manifesto's, quoted exactly

Theme 1 · Customer and value

- 1 Early, continuous delivery
- 2 Welcome late change
- 4 Business and developers daily

Theme 2 · People and interaction

- 5 Motivated, trusted people
- 6 Face-to-face conversation
- 11 Self-organizing teams

Theme 3 · Working product and feedback

- 3 Deliver frequently
- 7 Working software is progress
- 12 Reflect and adjust

Theme 4 · Sustainability and excellence

- 8 Sustainable pace
- 9 Technical excellence
- 10 Simplicity



0:00 · the 60-minute plan · 1:00 — the rust block is where we are



Theme 1 · Customer and value

Lesson 3 · 0:30–0:40

1

Principle 1. *Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.*

What you can observe: Something usable reaches a user in the first weeks, and again at short intervals — not once at the end

2

Principle 2. *Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.*

What you can observe: A change request is re-ordered into the work as information, not refused because it is late

4

Principle 4. *Business people and developers must work together daily throughout the project.*

What you can observe: The people who know the need and the people who build talk every day — not through a document once a quarter

The same idea from three sides: the customer's need is the reason the work exists, and the team keeps a continuous connection to it.



0:00 · the 60-minute plan · 1:00 — the rust block is where we are



Theme 2 · People and interaction

Lesson 3 · 0:30–0:40

5

Principle 5. *Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.*

What you can observe: The team decides how to do its work; management removes obstacles instead of assigning tasks

6

Principle 6. *The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.*

What you can observe: A question is asked across the table, or on a video call, before it is written in a ticket

11

Principle 11. *The best architectures, requirements, and designs emerge from self-organizing teams.*

What you can observe: Design decisions are made by the people who will build and maintain the design, and may change as the team learns

Note the word "emerge": the design is not fixed in advance — it takes shape as the work proceeds.



0:00 · the 60-minute plan · 1:00 — the rust block is where we are



Theme 3 · Working product and feedback

Lesson 3 · 0:30–0:40

3

Principle 3. *Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.*

What you can observe: A fixed, short delivery interval — in this course, two weeks

7

Principle 7. *Working software is the primary measure of progress.*

What you can observe: The team reports what a user can now do — not a percentage of a plan completed

12

Principle 12. *At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.*

What you can observe: A retrospective at the end of every interval, with at least one change actually made before the next one

The three form a loop: deliver, measure by what works, reflect, adjust.



0:00 · the 60-minute plan · 1:00 — the rust block is where we are



Theme 4 · Sustainability and excellence

Lesson 3 · 0:30–0:40

8

Principle 8. *Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.*

What you can observe: No "crunch": the work taken into an interval is what the team finished in the previous ones

9

Principle 9. *Continuous attention to technical excellence and good design enhances agility.*

What you can observe: Tests are written, code is reviewed, shortcuts are recorded and paid back

10

Principle 10. *Simplicity — the art of maximizing the amount of work not done — is essential.*

What you can observe: Build the smallest thing that answers the need; leave out what nobody asked for

The twelve principles are © 2001 the Manifesto's authors — full text: agilemanifesto.org/principles.html



0:00 · the 60-minute plan · 1:00 — the rust block is where we are



Iteration, increment, feedback loop

Lesson 3 · 0:30–0:40



Iteration

the **CYCLE**

One complete pass through plan → build → verify → review, of fixed length, repeated



Increment

the **OUTPUT**

The usable piece a cycle adds to the product — added to everything delivered before and verified together with it



Feedback loop

the **CORRECTION**

From control engineering: the output is measured, compared with the goal, and the difference corrects the next input — here: increment → a real user tries it → the remaining work changes

The test: a cycle repeated without adding anything usable = **refining**; pieces added without a fixed cycle = **accumulating** without a review point. Agile methods do both at once — and without the loop, an iteration is just a schedule.

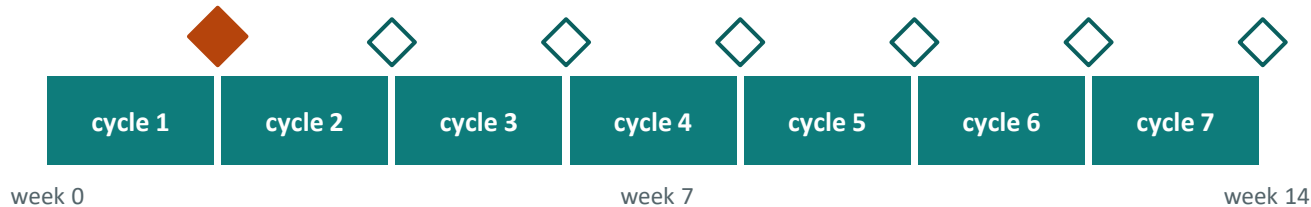


0:00 · the 60-minute plan · 1:00 — the rust block is where we are



Why the cycle must be short — "fail fast, learn faster"

Short cycle — a real review every two weeks



Wrong assumption found at the first review, week 2: two weeks of work built on it
→ a change of plan

Long cycle — one review, at the end



Wrong assumption found at the only review, week 14: fourteen weeks of work built on it → a failure of the plan

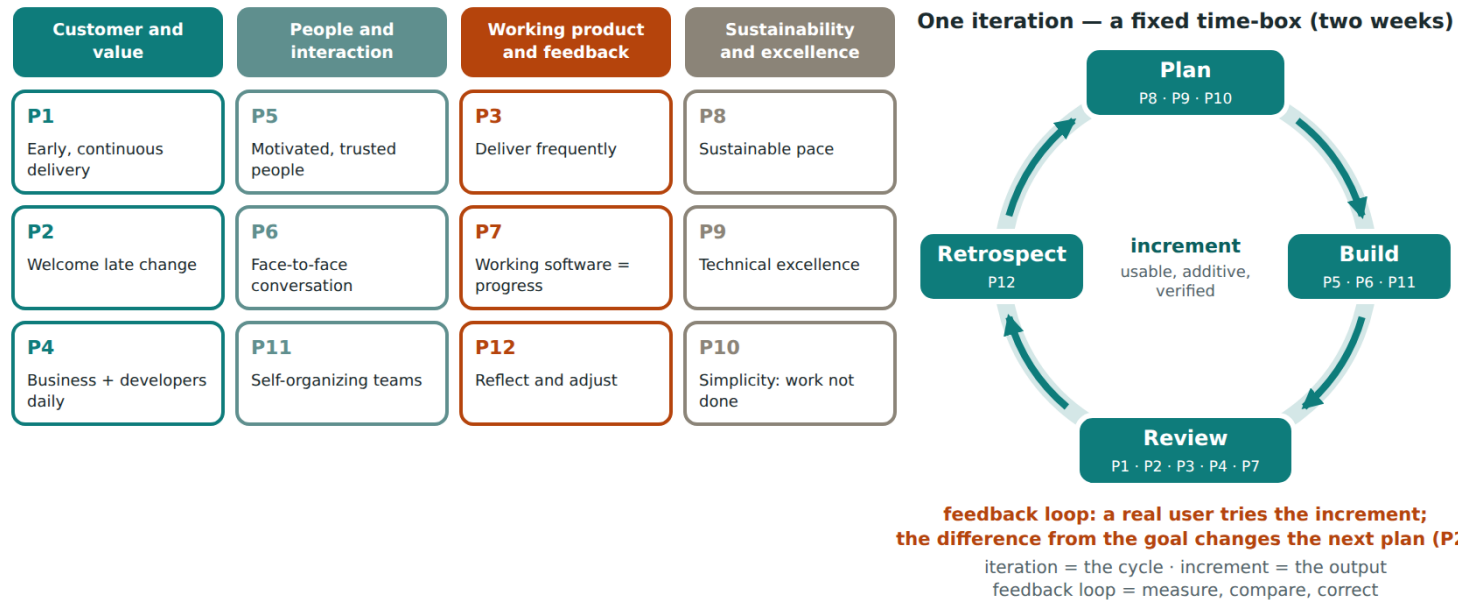
Illustration · white diamond = a review with a real user · rust diamond = the review that finds it

- Every plan contains assumptions that are wrong
- The cost of discovering one grows with the amount of work built on top of it
- A short cycle with a real review finds it while little is built on it: a **change of plan**, not a failure of the plan
- The value is not in failing — it is the **low cost** of the discovery and the **speed** of the correction
- Lesson 1 from the other side: information narrows the cone, and a two-week review produces information every two weeks

The container: the Sprint

Twelve principles in four themes — and the cycle that makes them possible

Principles © 2001 the Manifesto authors (agilemanifesto.org/principles.html), shortened here to their observable behaviour; the four themes are this course's grouping.



Six of the twelve principles cannot be practised without a short repeating cycle; in Scrum the cycle is the Sprint (Class 3).

Retrospect closes the loop: one change made before the next cycle begins (P12).

- **Sprint:** Scrum's iteration — a fixed time-box of one month or less (two weeks here)
- Plan · build · review with stakeholders · retrospective
- Where the principles sit:
 - 1 · 3 · 7 — the increment at the Sprint's end
 - 2 · 4 — the Product Owner in the review
 - 5 · 6 · 11 — the Developers deciding how
 - 12 — the Retrospective
 - 8 · 9 · 10 — Definition of Done, amount of work
- Next Monday we open Scrum

Activity · Fun-quiz ×3: which principle is broken? (5 min)

- Hands up, no phones — for each situation: name the broken principle and its theme

1. The team has not spoken to a customer in four months.
2. Nobody may change the plan once it is signed.
3. The team works seventy-hour weeks "until launch".

- One minute per situation: 30 seconds for hands and the answer, 30 seconds for the reason
- Taking part earns the activity points; the answers are never graded

5

minutes

0:40 – 0:45

taking part earns activity points

Videos for this lesson (each under 15 minutes):

- **Video:** [12 principles of Agile Software Development | Agile Principles Explained | Agilemania](https://www.youtube.com/watch?v=UzRs36B_8ko) youtube.com/watch?v=UzRs36B_8ko
- **Video:** [Incremental & iterative: a real-life example project](https://www.youtube.com/watch?v=ITOWKUjr8VA) youtube.com/watch?v=ITOWKUjr8VA



Real-life example 3 · One Sprint against the twelve principles

Lesson 3 · 0:30–0:40

- Team Menu's first two-week Sprint: ten days, fourteen events
- Each event supports or breaks principles: the cafeteria manager at planning (4, 5, 11) · skipped tests (breaks 9) · a missed stand-up (breaks 6) · overtime (breaks 8)
- Tiles turn green, amber or rust as evidence accumulates; the increment panel fills as features become usable
- Result: three usable features; "Print the menu" started, not usable, not asked for

The twelve principles, coloured by the evidence of the Sprint — and the increment

Customer and value	People and interaction	Working product and feedback	Sustainability and excellence	Increment (usable, additive)
P1 Early, continuous delivery ✓ 2 ✗ 0	P5 Motivated, trusted people ✓ 1 ✗ 0	P3 Deliver frequently ✓ 2 ✗ 0	P8 Sustainable pace ✓ 0 ✗ 1	Today's menu usable since day 4
P2 Welcome late change ✓ 2 ✗ 0	P6 Face-to-face conversation ✓ 2 ✗ 1	P7 Working software is progress ✓ 4 ✗ 0	P9 Technical excellence ✓ 1 ✗ 1	Allergen labels usable since day 7
P4 Business and developers daily ✓ 2 ✗ 0	P11 Self-organizing teams ✓ 2 ✗ 0	P12 Reflect and adjust ✓ 1 ✗ 0	P10 Simplicity ✓ 2 ✗ 1	Tomorrow's menu usable since day 9
				Print the menu started, not usable

green = supported more than broken · rust = broken more than supported · amber = equal · sand = no evidence yet
✓ = events supporting the principle · ✗ = events breaking it · a hatched feature was started but is not part of the increment

Final state of the simulation · run it live in the lecture page (Start / Pause · Next step · Restart · Copy)

Where it returns in the labs: **D4** (Deliverable 4) Daily Scrum log and Definition of Done · **D5** (Deliverable 5) retrospective checklist · **CQ1** (Checkpoint Quiz 1)



0:00 · the 60-minute plan · 1:00 — the rust block is where we are



Lesson 3 in six cards

Lesson 3 · 0:30–0:40

Name the four themes and one principle in each.

Customer and value (1, 2, 4) · People and interaction (5, 6, 11) · Working product and feedback (3, 7, 12) · Sustainability and excellence (8, 9, 10).

Principle 7, in your own words?

Working software is the primary measure of progress: report what a user can now do, not a percentage of a plan.

Define iteration.

One complete pass of plan → build → verify → review, of fixed length, repeated — the cycle.

Define increment.

The usable, additive result a cycle adds to the product, verified with everything delivered before — the output.

What is a feedback loop, applied to a product?

The increment is measured by a real user trying it; the difference from the goal changes the order or content of the remaining work.

What does "fail fast, learn faster" mean precisely?

Discover wrong assumptions while little is built on them: the value is the low cost of the discovery and the speed of the correction, not the failure.



0:00 · the 60-minute plan · 1:00 — the rust block is where we are



Lesson 4 · Denning's three laws

0:45 – 0:55 · then the activity: zen-break + one-minute paper (5 min)

What the organizations that made Agile work at scale have in common — the three laws of the required text — and why the same laws apply to a hotel, a bank, a marketing department and a school.

✓	L1	act	L2	act	L3	act	L4	act
---	----	-----	----	-----	----	-----	----	-----

0:00 · the 60-minute plan · 1:00 — the rust blocks are this lesson and its activity



Denning's question

- The Manifesto (2001) was written by and for software developers
- The required text — Stephen Denning, *The Age of Agile* (2018) — asks: when the same ideas were applied to whole organizations, what did the ones that succeeded have in common?
- Banks, manufacturers, retailers, a music service, a fighter-aircraft programme
- Denning: former director of knowledge management at the World Bank, author on leadership, Forbes columnist
- His answer: three regularities he calls **laws** — present wherever Agile worked, absent where it failed

Law of the Small Team

Chapter 2 — this week's reading

Law of the Customer

Chapter 3

Law of the Network

Chapter 4



The Law of the Small Team (1/2)

Lesson 4 · 0:45–0:55

Denning's statement: *Work should in principle be done by small, autonomous, cross-functional teams, working in short cycles on relatively small tasks, and obtaining continuous feedback from the ultimate customer or end user.*

Small

everyone knows what everyone else is doing — Amazon's rule: a team two pizzas can feed

Autonomous

the team decides how to do the work: it is given a goal, not a task list

Cross-functional

every skill needed to finish the work, without waiting for another department

Short cycles

the iteration of Lesson 3

Small tasks

work cut into pieces that fit inside one cycle

Continuous feedback

from the end user: the feedback loop, closed every cycle



0:00 · the 60-minute plan · 1:00 — the rust block is where we are



The Law of the Small Team (2/2)

Why it is called a law

- Remove any one condition and the results of the others disappear with it:
 - a team of forty → no longer **small**
 - permission needed for every decision → no longer **autonomous**
 - no tester → no longer **cross-functional**
 - a cycle of six months → no longer **short cycles**

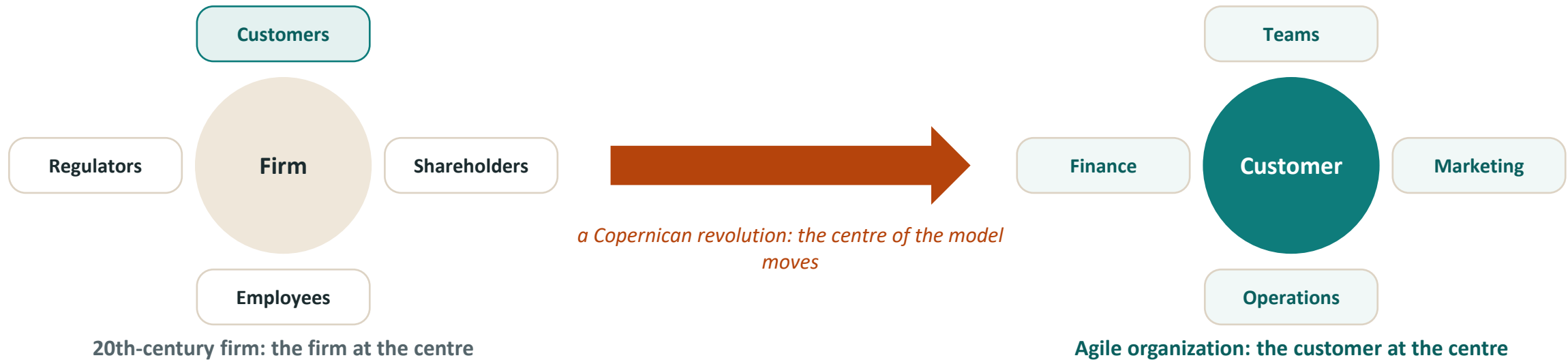
Menlo Innovations — Chapter 2

- A software company in Ann Arbor, Michigan
- Pairs that change every week · one-week cycles · clients and users in the room
- Denning's reason for choosing it: the law operating at its simplest scale
- menloinnovations.com



The Law of the Customer

Denning's statement: *The organization is obsessed with delivering value to customers — understood as the purpose of the whole organization, not as a slogan of the sales department.*



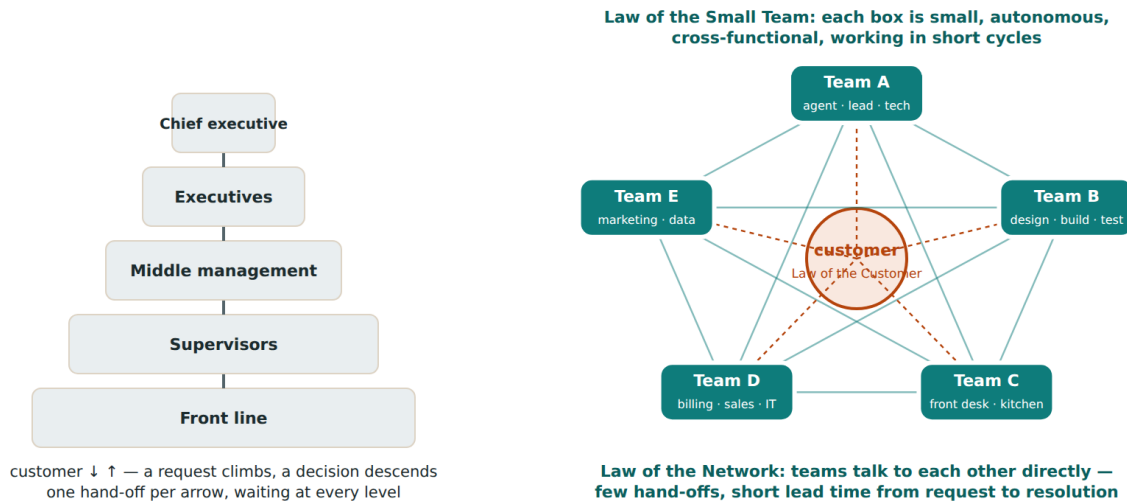
- Denning's **Copernican revolution** in management: the centre of the model moves
- Peter Drucker (1954): the only valid definition of a business's purpose is to create a customer

- Why now: globalization, deregulation and the Internet moved market power to the buyer — compare in seconds, switch in one click
- Principle 1 is this law seen from inside one team

The Law of the Network

Pyramid or network — Denning's three laws in one picture

The Age of Agile (2018), Chapters 2–4. Left: a top-down bureaucracy.
Right: small, autonomous, cross-functional teams connected directly, around the customer.



Left: information rises through layers, decisions descend. Efficient in a stable environment; late in a VUCA one.

Right: the Small Team (each box), the Customer (the centre), the Network (the direct lines) — needed together.

- A transparent **network of teams** collaborating toward the customer — not a top-down bureaucracy where information rises and decisions descend
- Why it exists: excellent small teams inside an unchanged hierarchy were absorbed or dissolved within two or three years
- Hierarchy does not vanish — someone still decides the strategy; requests travel directly between the teams that need them
- Spotify: squads, tribes, chapters, guilds · Barclays: thousands of people in such teams
- Measurable sign: hand-offs and days from request to resolution — the **lead time** of Class 6

How the three laws fit together

Lesson 4 · 0:45–0:55

WHERE

The Law of the

Small Team

where the work is done

+

WHY

The Law of the

Customer

why it is done

+

HOW

The Law of the

Network

how the teams are connected
so that the first two survive

- **Needed together:** small teams without customer focus optimize themselves; customer focus without small teams cannot deliver; both without a network are crushed by the bureaucracy around them
- **VUCA** (Volatility, Uncertainty, Complexity, Ambiguity) — a term of the United States Army War College: in a stable environment a bureaucracy is efficient; in a VUCA environment it is late



0:00 · the 60-minute plan · 1:00 — the rust block is where we are



Agile beyond software

Lesson 4 · 0:45–0:55

The Manifesto says "software" in every value; the three laws say nothing about software at all.

MARKETING

Agile Marketing Manifesto — San Francisco, June 2012

Validated learning over opinions
and conventions · many small
experiments over a few large bets ·
adaptive campaigns over big-bang
campaigns

agilemarketingmanifesto.org

EDUCATION

eduScrum — Willy Wijnands, the Netherlands, 2011

Student teams plan, build and
review their learning in sprints; the
teacher is the Product Owner

www.eduscrum.org

HOSPITALITY

The reference sector of your program

A guest's Wi-Fi complaint: three
days up and down a chain of
command — or one hour with a
small cross-functional team that
has its own authority

Next Monday: Scrum — the roles, events and artifacts of the Small Team — and **CQ1** (Checkpoint Quiz 1) on the Manifesto, the principles and the three laws.



0:00 · the 60-minute plan · 1:00 — the rust block is where we are



Activity · Zen-break + one-minute paper (5 min)

- 90 seconds: stand up, stretch, no talking, no phones — breathe slowly, look away from every screen
- Then write one full sentence:
 - "The law my product needs most is _____ because _____."
- Name one of the three laws; give a reason specific to **your** team's product idea
- 60 seconds to write · hand the card in on the way to the studio
- It comes back with a comment — and, expanded to a paragraph, it closes your **D2** (Deliverable 2)

5

minutes

0:55 – 1:00

taking part earns activity points

Videos for this lesson (each under 15 minutes):

- **Video:** [Spotify Engineering Culture - Part 1 \(aka the "Spotify Model"\)](https://youtube.com/watch?v=Yvfz4HGtoPc) youtube.com/watch?v=Yvfz4HGtoPc
- **Video:** [What is Agile Project Management? Project Management in Under 5](https://youtube.com/watch?v=D5FoRXGa8ic) youtube.com/watch?v=D5FoRXGa8ic



0:00 · the 60-minute plan · 1:00 — the rust block is where we are



Real-life example 4 · Hôtel du Plateau — pyramid or network

Lesson 4 · 0:45–0:55

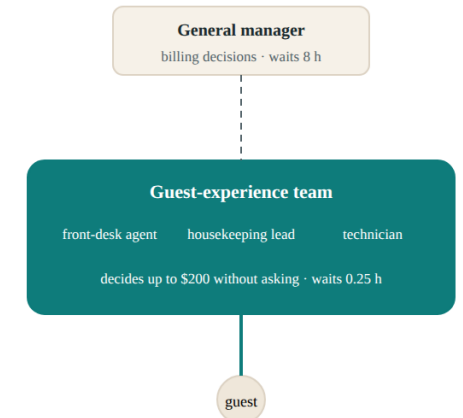
- A forty-room hotel in Montréal receives six guest requests in one week
- Pyramid: front desk → duty manager → general manager → external contractor, and back down — a hand-off and a wait at every level
- Network: one guest-experience team (agent, housekeeping lead, technician) that may spend \$200 without asking
- Average lead time: 14.7 h against 2.0 h — about seven times faster, same people, same skills

Where does a guest's request go? Left: the pyramid. Right: one team with authority.

this request: 3 hand-offs, 2.90 h · average lead time so far 14.7 h



this request: 1 hand-offs, 0.65 h · average lead time so far 2.0 h



rust = where this request was resolved · dotted line = escalation, used only without authority

Final state of the simulation · run it live in the lecture page (Start / Pause · Next step · Restart · Copy)

Where it returns in the labs: D2 (Deliverable 2) closing paragraph · D6 (Deliverable 6) lead time · D7 (Deliverable 7) organization map · Mid-Term Part D



0:00 · the 60-minute plan · 1:00 — the rust block is where we are



Lesson 4 in six cards

Lesson 4 · 0:45–0:55

State the Law of the Small Team.

Work is done by small, autonomous, cross-functional teams, in short cycles, on relatively small tasks, with continuous feedback from the end user (Denning, Chapter 2).

State the Law of the Customer.

Delivering value to the customer is the purpose of the whole organization — the customer at the centre; Denning's "Copernican revolution" in management (Chapter 3).

State the Law of the Network.

The organization works as a transparent network of teams collaborating toward the customer, not as a top-down bureaucracy in which information rises and decisions descend (Chapter 4).

Why are the three laws needed together?

Small teams without customer focus optimize themselves; customer focus without small teams cannot deliver; both without a network are crushed by the surrounding bureaucracy.

What does VUCA stand for, and why does it matter here?

Volatility, Uncertainty, Complexity, Ambiguity — in a stable environment a bureaucracy is efficient; in a VUCA environment it is late.

Name two documented applications of Agile outside software.

The Agile Marketing Manifesto (San Francisco, 2012) and eduScrum (Willy Wijnands, the Netherlands, 2011) — and the three laws applied to a hotel, a bank or any service.



After the hour — Team Studio, Home Sprint and D2

Team Studio • 120 min

Goal: Product Vision v1 and an Agile fit check

- Planning (10)
- Product vision — elevator-pitch template (40)
- Two plans, one product (30)
- Manifesto map (20)
- Sprint Review: 90-second vision pitches (15)
- Retrospective (5)

Home Sprint • 120 min

Before next Monday

- Denning, Chapter 2: "The Law of the Small Team"
- Re-read Box 1-1 until you can recite the four values
- Finish D2 with your team; update the board
- Learning Log

D2 (Deliverable 2) — Product Vision Board

Due Sunday, October 4, 23:59 · Blackboard

- Vision, users and their top three needs
- The first three increments
- Manifesto map · Waterfall-vs-Agile comparison
- One paragraph on Denning's laws
- Mid-Term basis — keep your own copy

Next Monday — Class 3 · Scrum I: the framework, the pillars, the roles · **CQ1** (Checkpoint Quiz 1): 15 minutes, individual, on Classes 1 to 3



All links used in this lecture (1/2)

After the hour

[Manifesto for Agile Software Development](#)

The four values, exactly as written in February 2001, with the seventeen authors' names; Box 1-1 in Denning. Read in Lesson 2.

agilemanifesto.org/

[The twelve principles behind the Agile Manifesto](#)

The full text of the twelve principles grouped into four themes in Lesson 3.

agilemanifesto.org/principles.html

[History: the Agile Manifesto \(Snowbird, 2001\)](#)

Jim Highsmith's account of the meeting of February 11–13, 2001, written for the site by one of the seventeen.

agilemanifesto.org/history.html

[Agile Alliance](#)

The non-profit founded in 2001 by the Manifesto's authors; its Agile 101 pages define the terms used this week.

www.agilealliance.org/

[Construx — The Cone of Uncertainty](#)

Steve McConnell's reference description of the cone, with Boehm's factors — the source of the estimate ranges in Lesson 1.

www.construx.com/books/the-cone-of-uncertainty/

[The Standish Group — CHAOS reports](#)

The research firm whose CHAOS studies, published since 1994, measure how often software projects finish on time, on budget and in scope.

www.standishgroup.com/

[Steve Denning — official site](#)

The author of the required text; the site links his Forbes column, where the three laws were first published as an article in 2016.

www.stevedenning.com/

[Denning \(2016\), "Understanding the three laws of Agile"](#)

The peer-reviewed article that states the three laws; available through the Digital Learning Resource Center.

doi.org/10.1108/SL-09-2016-0074

[The Age of Agile — publisher page \(HarperCollins Leadership\)](#)

Publisher's page for the required text, Denning (2018). Your copy is in VitalSource; Chapter 2 is this week's reading.

www.harpercollinsleadership.com/9781400242405/the-age-of-agile

Before you open a link: check it with [VirusTotal](#) — Cyber.SoHo Legal Notice, section 14. Official sites only; third-party links are not endorsements.



All links used in this lecture (2/2)

After the hour

[Spotify Engineering — "Spotify Engineering Culture, Part 1"](#)

Henrik Kniberg's original post and video on squads, tribes, chapters and guilds — Denning's Chapter 4 example.

engineering.atspotify.com/2014/03/spotify-engineering-culture-part-1

[Menlo Innovations](#)

The Ann Arbor software company Denning uses in Chapter 2 to show the Law of the Small Team at its simplest scale.

menloinnovations.com/

[Agile Marketing Manifesto](#)

The 2012 manifesto written by marketers on the model of the 2001 text — Agile beyond software, in Lesson 4.

agilemarketingmanifesto.org/

[eduScrum](#)

Willy Wijnants's adaptation of Scrum for school classes (2011) — Agile in education, in Lesson 4.

www.eduscrum.org/

[The Scrum Guide \(scrumguides.org\)](#)

The official, free definition of Scrum — the Sprint named in Lesson 3 and the whole of next Monday's class; read it before Class 3 if you can.

scrumguides.org/

[VitalSource Bookshelf](#)

The e-textbook application where The Age of Agile is provided to you at no cost.

www.vitalsource.com/

[tutor.com](#)

24/7 tutoring and writing review — open it from inside the Blackboard course so that your use is counted; use it for your Learning Log.

www.tutor.com/

[VirusTotal — URL \(Uniform Resource Locator\) scanner](#)

Scan any link from this page before opening it, as the Legal Notice (section 14, Annex C) asks.

www.virustotal.com/gui/home/url

[Cyber.SoHo — Legal Notice, Terms of Use and Acceptable Use Policy](#)

The terms under which this material is provided to you: sections 4, 5, 14, 17, 18 and 20 apply to this page.

cyber-soho.github.io/prevmon/00.01.cyber.soho-legal.html

Before you open a link: check it with [VirusTotal](#) — Cyber.SoHo Legal Notice, section 14. Official sites only; third-party links are not endorsements.



Legal notice for this material

- These slides and the lecture module are Cyber.SoHo Materials — © 2026 Cyber.SoHo, all rights reserved
- Use is subject to the [Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy](#)
- §4 — one personal study copy; no re-posting, sharing, sale, recording, remixing or use as training data
- §5 — authors, vendors, publishers, the Agile Alliance and the university are named for identification only
- §14 — check every link before opening it: [VirusTotal](#)
- §17 — sessions may not be recorded, photographed or streamed · §18 — submitted work must be your own · §20 — personal information
- The Manifesto and its twelve principles are quoted in their entirety with their own notice (© 2001, their authors)
- The institution's own policies apply in addition; where they overlap, the stricter rule governs

